WALLET PROJESİ · ÇALIŞMA NOTLARI

JWT ile Kontrollü Giriş

Spring Boot backend ve Expo mobil uygulamada token tabanlı giriş, kayıt ve kullanıcıya özel harcamalar: neyi, neden, nasıl yaptık.

1 Büyük resim
2 JWT nedir, nasıl çalışır?
3 Şifreler: BCrypt ve salt
4 Backend: JWT altyapısı
5 Backend: giriş ve kayıt
6 Backend: sadece kendi harcamaların
7 Mobil: token nerede durur?
8 Mobil: istekler token ile nasıl gider?
9 Mobil: oturum, yönlendirme, ekranlar
10 Uçtan uca akışlar
11 Yolda karşılaşılan sorunlar
12 Bilinen sınırlar ve sonraki adımlar

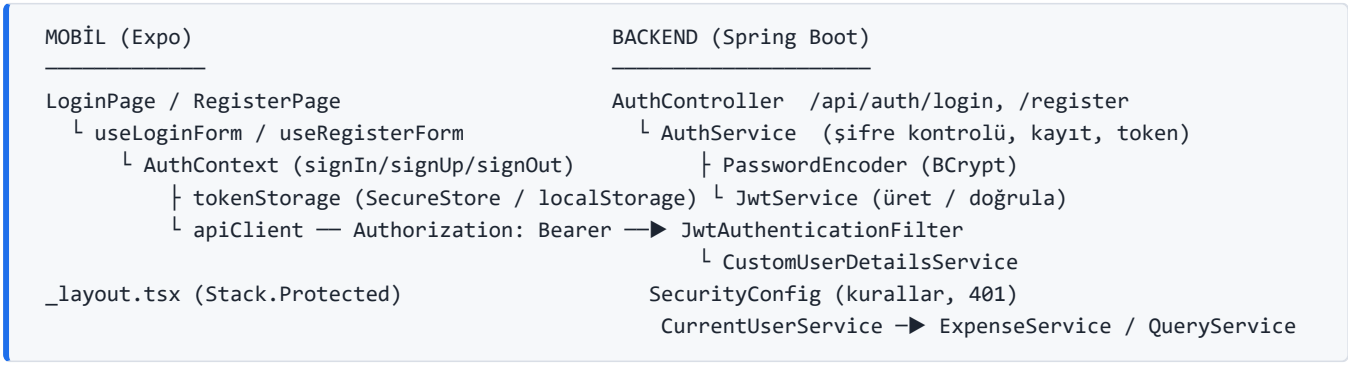
29 Ekim 2026 · Backend: Spring Boot 4.1.1, Spring Security 7, JJWT 0.12.6 · Mobil: Expo SDK 58, Expo Router

1 Büyük resim

Başlangıçta uygulamada gerçek bir giriş yoktu. Login ekranı sadece tasarımdan ibaretti, "Giriş yap" düğmesi doğrudan ana sayfayı açıyordu. Mobil uygulama her harcamayı sabit bir `CURRENT_USER_ID = 1` ile gönderiyor, backend de bu sayıya körü körüne güveniyordu. Liste ise herkesin harcamalarını döndürüyordu.

Konu	Önce	Şimdi
Giriş	Kontrol yok, düğme doğrudan /home'a gidiyor	Kullanıcı adı ve şifre backend'de doğrulanıyor, JWT dönüyor
Kayıt	Yok	Kayıt ekranı var, kayıttan sonra doğrudan oturum açılıyor
Şifre saklama	Kayıt akışı yoktu	BCrypt ile hash'lenmiş olarak saklanıyor
API erişimi	Herkese açık	`/api/auth/**` dışındaki her adres token istiyor, yoksa 401
Harcamanın sahibi	İstemcinin gönderdiği `userId`	Token'daki kullanıcı; istekteki `userId` yok sayılıyor
Harcama listesi	Herkesin harcamaları	Sadece giriş yapan kullanıcınınkiler
Oturum (mobil)	Yok	Token güvenli depoda; açılışta okunuyor, 401'de ve çıkışta siliniyor

Parçalar bir arada



2 JWT nedir, nasıl çalışır?

JWT (JSON Web Token), sunucunun "bu kişi şudur" diye imzaladığı küçük bir metindir. Kullanıcı bir kez şifreyle giriş yapar, sunucu ona token verir, sonraki her istekte kullanıcı bu token'ı gösterir.

1	**Mobil**	`POST /api/auth/login` ile kullanıcı adı ve şifreyi gönderir.
2	**Backend**	Şifreyi kontrol eder. Doğruysa imzalı bir token üretip döner.
3	**Mobil**	Token'ı telefonun güvenli deposuna kaydeder.
4	**Mobil**	Sonraki her isteğe `Authorization: Bearer <token>` başlığını ekler.
5	**Backend**	İmzayı ve süreyi kontrol eder. Geçerliyse isteği kabul eder, değilse **401** döner.

Token'ın üç parçası

eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJtdXJhdCIsImlhdCI6MTc5MzI3MDQwMCwiZXhwIjoxNzkzMzU2ODAwfQ.-MOAfLIHUDupO7lmmnwJf0_OihMWQxOFQvKXwwlpeFw

Parça	İçerik (Base64'ten çözülünce)	Görevi
Header	`{"alg":"HS256"}`	İmzanın hangi algoritmayla atıldığını söyler.
Payload	`{"sub":"murat","iat":…,"exp":…}`	`sub`: kime ait · `iat`: üretilme zamanı · `exp`: bitiş zamanı (bizde 24 saat). **Şifreli değildir**, herkes okuyabilir.
Signature	`HMAC-SHA256(header.payload, gizli anahtar)`	Header ve payload'ın değiştirilmediğini kanıtlar. Gizli anahtar olmadan üretilemez.

> **Neden güvenli?**
> Biri payload'daki `"sub":"murat"` değerini `"admin"` yaparsa imza artık tutmaz: sunucu imzayı kendi anahtarıyla yeniden hesaplar, sonuç farklı çıkar ve istek reddedilir. Benzetmeyle: payload kimlik kartı, imza hologram. Kartı herkes okuyabilir ama hologramı sadece sunucu basabilir.

Gizli anahtar (secret)

- Rastgele üretilmiş **32 bayt (256 bit)**, yaml dosyasında Base64 metin olarak duruyor. JJWT, HS256 için daha kısa bir anahtarı kabul etmez.
- PC'ye değil uygulamanın ayarına bağlıdır. Sen değiştirmedikçe sabit kalır. Değişirse eski token'ların hepsi geçersiz olur.
- Sadece sunucuda durur, mobil uygulama bilmez. Canlı ortamda `JWT_SECRET` ortam değişkeninden gelmeli, git'te durmamalı.

```
PowerShell ile üretmek:
[Convert]::ToBase64String((1..32 | ForEach-Object { Get-Random -Maximum 256 }))
```

401 ve 403 farkı

Kod	Anlamı	Bizde ne zaman
401	"Seni tanımıyorum" (kimlik doğrulama)	Token yok, süresi dolmuş, imzası bozuk; login'de yanlış şifre
403	"Tanıyorum ama yetkin yok" (yetkilendirme)	Şu an kullanılmıyor. Rol eklenirse kullanılır.

3 Şifreler: BCrypt ve salt

Şifre veritabanına **asla düz metin** olarak yazılmaz; BCrypt ile hash'lenir. Hash tek yönlüdür, hash'ten şifreye geri dönülemez. Girişte yapılan şey "girilen şifreyi aynı yöntemle hash'le ve kayıtlı hash'le karşılaştır" kontrolüdür: `passwordEncoder.matches(düzŞifre, hash)`.

Soru: İki kullanıcımın şifresi aynı ama veritabanındaki hash'leri farklı. Neden?

Bu beklenen ve doğru bir davranış. BCrypt her `encode` çağrısında rastgele bir **salt** üretir ve şifreyi bu salt ile birlikte hash'ler. Salt hash'in içine yazıldığı için ayrıca saklanmaz:

```
$2a$10$N9qo8uLOickgx2ZMRZoMye IjZAgcfl7p92ldGxad68LJZdL17lhWy
└┘└┘└┘└────────────────────┘└──────────────────────────────┘
 │  │  │      salt (22 karakter)         asıl hash (31 karakter)
 │  │  └ güç (cost): 2^10 = 1.024 tur
 │  └ algoritma sürümü (BCrypt)
(Salt ile hash arasındaki boşluk sadece göstermek için; gerçek değer tek parça.)
```

`matches("123456", hash)` çalışırken salt'ı hash'in içinden okur, girilen şifreyi o salt ile yeniden hash'ler ve sonucu karşılaştırır. Bu yüzden iki farklı hash de `123456` için `true` döner.

Neden böyle? Salt olmasaydı aynı şifreler aynı hash'i üretirdi. Veritabanı sızdığında saldırgan "bu iki kişinin şifresi aynı" diye anlayabilir, önceden hesaplanmış tablolarla (rainbow table) yaygın şifreleri tek seferde çözebilirdi. Salt her kullanıcıyı ayrı ayrı kırmaya zorlar.

Güç (strength / cost) değeri

`new BCryptPasswordEncoder()` varsayılan olarak **10** kullanır: 2^{10} tur, yaklaşık 0,1 saniye. Değer bir artınca süre iki katına çıkar. BCrypt'in **bilerek yavaş** olması kullanıcıyı rahatsız etmez, ama milyarlarca deneme yapan saldırganı çok yavaşlatır.

Bizde nerede kullanılıyor

Yer	Ne yapıyor
`SecurityConfig.passwordEncoder()`	`@Bean` olarak tek bir `BCryptPasswordEncoder` üretir; isteyen sınıfa Spring verir.
`AuthService.register()`	`passwordEncoder.encode(şifre)`, hash'i kaydeder.
`AuthService.login()`	`passwordEncoder.matches(girilen, kayıtlı)`: sıra önemli, önce düz şifre.

4 Backend: JWT altyapısı

4.1 Bağımlılıklar ve ayarlar

`pom.xml`: `spring-boot-starter-security` ve JJWT'nin üç paketi (`jjwt-api`, `jjwt-impl` ve `jjwt-jackson`; son ikisi `runtime` kapsamında). Security eklendiği anda Spring **her adresi kilitler** ve konsola geçici bir şifre yazar. Kendi ayarlarımızı yazınca bu davranış kalktı.

`wallet-be/src/main/resources/application.yaml`

```
app:
  jwt:
    secret: ${JWT_SECRET:<base64 anahtar>}   # ortam değişkeni varsa o, yoksa bu
    expiration-ms: 86400000                  # 24 saat
```

4.2 JwtService: token üret ve oku

`security/JwtService.java`

```
public JwtService(@Value("${app.jwt.secret}") String secret, @Value("${app.jwt.expiration-ms}") long
expirationMs) {
    this.key = Keys.hmacShaKeyFor(Decoders.BASE64.decode(secret));   // metin → 32 bayt → imza anahtarı
    this.expirationMs = expirationMs;
}

public String generateToken(String userName) {
    Date now = new Date();
    return Jwts.builder()
            .subject(userName)                                      // "sub"
            .issuedAt(now)                                          // "iat"
            .expiration(new Date(now.getTime() + expirationMs))     // "exp"
            .signWith(key)                                          // imza
            .compact();                                             // xxxxx.yyyyy.zzzzz
}

// İmza bozuksa veya süresi dolmuşsa JwtException fırlatır
public String extractUserName(String token) {
    return Jwts.parser().verifyWith(key).build()
            .parseSignedClaims(token).getPayload().getSubject();
}
```

4.3 CustomUserDetailsService: Spring ile veritabanı arasındaki köprü

Spring Security kullanıcıların `users` tablosunda olduğunu bilmez. `UserDetailsService` arayüzünü uygulayan bu sınıf, kullanıcı adını alır, `findByUserName` ile bulur ve Spring'in istediği dört bilgiyi döner: kullanıcı adı, şifre hash'i, yetkiler (şimdilik boş) ve hesabın açık mı kapalı mı olduğu (`!active`). Şifre kontrolü **yapmaz**, sadece "bu kişi kim?" sorusunu cevaplar. `implements UserDetailsService` yazılmazsa Spring sınıfı tanımaz.

> İsim çakışması: Spring'in de `User` adında bir sınıfı var. Bizim entity import edildi, Spring'inki tam adıyla yazıldı: `org.springframework.security.core.userdetails.User.builder()`.

4.4 JwtAuthenticationFilter: her isteğin kapısındaki kontrol

1. `Authorization` başlığını oku. Yoksa ya da `"Bearer "` ile başlamıyorsa **hiçbir şey yapmadan** devam et (login isteği böyle geçer).
2. Baştaki `"Bearer "` kısmını at, kalan metni `jwtService.extractUserName(token)` ile doğrula.
3. `loadUserByUsername`: kullanıcı hâlâ var mı, hesabı açık mı?
4. Geçerliyse `SecurityContextHolder` kutusuna kullanıcıyı koy. Bundan sonra Spring "bu istek murat'tan" bilir.
5. Hata olursa (`JwtException`, `UsernameNotFoundException`) kutu boş kalır. **Filtre 401 dönmez**; kararı SecurityConfig verir.

`OncePerRequestFilter` filtrenin her istekte tam bir kez çalışmasını garanti eder. `filterChain.doFilter(...)` çağrılmazsa istek controller'a hiç ulaşmaz.

4.5 SecurityConfig: kurallar listesi

config/SecurityConfig.java

```
http
    .csrf(csrf -> csrf.disable())
    .cors(Customizer.withDefaults())
    .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
    .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**").permitAll()
            .requestMatchers("/error").permitAll()
            .anyRequest().authenticated())
    .exceptionHandling(e -> e.authenticationEntryPoint(new
HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED)))
    .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);
```

Kural	Neden
CSRF kapalı	CSRF saldırısı tarayıcının cookie'yi otomatik eklemesine dayanır. Biz cookie kullanmıyoruz, token'ı uygulama başlığa kendisi ekliyor.
CORS varsayılan	`WebConfig`'teki ayarı kullanır. Expo web sürümü farklı bir adresten istek atıyor.
STATELESS	Sunucuda oturum ya da cookie yok; kimlik her istekte token'dan okunur.
`/api/auth/**` açık	Token almak için token gerekmesin (login, register).
`/error` açık	Gerçek hatalar (500 gibi) 401'in arkasına gizlenmesin.
`anyRequest().authenticated()`	Geri kalan her şey kimlik ister. **Sıra önemli**: ilk eşleşen kural uygulanır, genel kural en sonda.
EntryPoint → 401	Kimliksiz isteğe her zaman 401 dönsün (aksi halde bazen 403). Mobil taraf 401'i görünce login'e gider.
`addFilterBefore`	Bizim filtre, yetki kontrolünden önce kimlik kutusunu doldursun.

5 Backend: giriş ve kayıt

Dosya	Görevi
`dto/auth/LoginRequest`	`record(userName, password)`, ikisi de `@NotBlank`
`dto/auth/RegisterRequest`	Ad ve soyad ≤50, kullanıcı adı 3–50, e-posta `@Email` ≤100, şifre 6–100 (entity kolon uzunluklarıyla aynı)
`dto/auth/AuthResponse`	`record(token)`
`UserRepository`	`findByUserName`, `existsByUserName`, `existsByEmail`; sorguları Spring Data üretir
`service/AuthService`	Giriş: kullanıcıyı bul, aktif mi ve şifre doğru mu bak, token üret. Kayıt: tekrar kontrolü, hash'le, kaydet, token üret.
`controller/AuthController`	`POST /api/auth/login` → 200 · `POST /api/auth/register` → 201
`GlobalExceptionHandler`	`BadCredentialsException` → **401** (bu handler olmasa 500 dönerdi)

`service/AuthService.java · login`

```
User user = userRepository.findByUserName(request.userName())
        .orElseThrow(() -> new BadCredentialsException(INVALID_CREDENTIALS));

if (!user.isActive() || !passwordEncoder.matches(request.password(), user.getPassword())) {
    throw new BadCredentialsException(INVALID_CREDENTIALS);
}
return new AuthResponse(jwtService.generateToken(user.getUserName()));
```

Güvenlik ayrıntıları

- **Aynı hata mesajı:** kullanıcı yoksa da, şifre yanlışsa da `"Invalid username or password"` dönüyor. Ayrı mesajlar saldırgana hangi kullanıcı adlarının kayıtlı olduğunu öğretirdi.
- **Loglama yok:** `LoginRequest` record'unun otomatik `toString` metodu şifreyi de yazar. Bu yüzden auth isteği loglanmıyor.
- **Kayıttan sonra token:** kullanıcı ayrıca giriş yapmak zorunda kalmıyor.

Denenen sonuçlar

Senaryo	Sonuç
Token'sız `GET /api/expenses`	401
Olmayan kullanıcıyla giriş	401
Boş alanlarla giriş	400 + alan hataları
Bozuk e-posta ya da kısa şifreyle kayıt	400 + alan hataları
Sahte imzalı token	401

6 Backend: sadece kendi harcamaların

Sorun şuydu: harcamanın sahibi istek gövdesindeki `userId` değerinden alınıyordu. İsteği elle atan biri `"userId": 5` yazarak başkası adına kayıt ekleyebilir, liste de herkesin verisini döndürüyordu. Oysa kullanıcının kim olduğu bilgisi zaten token'da duruyor.

```
Önce:   mobil "ben 1 numarayım" der          → backend inanır
Şimdi:  backend token'a bakar → "sen murat'sın, id'n 7" der
```

6.1 Kullanıcı nasıl bulunuyor: CurrentUserService

`security/CurrentUserService.java`

```
public User getCurrentUser() {
    Authentication authentication = SecurityContextHolder.getContext().getAuthentication();   //
filtrenin koyduğu kimlik
    ...
    return userRepository.findByUserName(authentication.getName())    // token'daki "sub"
            .orElseThrow(...);
}
```

1. İstek `Authorization: Bearer eyJ…` ile gelir.
2. **JwtAuthenticationFilter** token'dan `sub = "murat"` okur ve kimliği `SecurityContextHolder`'a koyar.
3. Servis katmanında **CurrentUserService** bu kutudan adı alır, `findByUserName("murat")` ile `User` entity'sine (id: 7) ulaşır.

6.2 Harcama eklendiğinde

`service/impl/ExpenseServiceImpl.java`

```
Expense expense = expenseMapper.toEntity(expenseDTO);          // mapper "user" alanını zaten kopyalamaz
expense.setUser(currentUserService.getCurrentUser());         // sahip = token'daki kullanıcı
return expenseMapper.toDto(expenseRepository.save(expense));
```

İstekte `userId` gönderilse bile (ör. 999) yok sayılır. Cevaptaki `userId` her zaman gerçek sahibin id'sidir.

6.3 Tek kayıt, güncelleme, silme

`repository/ExpenseRepository.java`

```
Optional<Expense> findByIdAndUserId(Long id, Long userId);   // "UserId" → expense.user.id
boolean existsByIdAndUserId(Long id, Long userId);
```

Getirme, güncelleme, kısmi güncelleme ve silme işlemleri bu metotlarla **"id'si X olan ve bana ait olan"** harcamayı arar. Başkasının harcaması bulunamaz ve **404** döner. 403 dönmüyoruz, çünkü 403 o id'de bir kayıt olduğunu ele verirdi.

6.4 Listeleme ve sayım

`service/query/ExpenseQueryService.java`

```
private Specification<Expense> createOwnSpecification(ExpenseCriteria criteria) {
    Long userId = currentUserService.getCurrentUser().getId();
    Specification<Expense> ownedByCurrentUser =
            (root, query, cb) -> cb.equal(root.get(Expense_.user).get(BaseEntity_.id), userId);
    return createSpecification(criteria).and(ownedByCurrentUser);   // istemcinin filtreleri VE "benim"
}
```

Kullanıcı hangi filtreyi gönderirse göndersin (kategori, tarih, tutar...), SQL'e her zaman `WHERE user_id = <benim id'm>` koşulu eklenir. Artık anlamı kalmayan `?userId.equals=` filtresi `ExpenseCriteria`'dan kaldırıldı. `ExpenseDTO.userId` artık zorunlu değil: cevapta dönüyor, istekte yok sayılıyor.

7 Mobil: token nerede durur?

Ortam	Depo	Neden / risk
iOS	`expo-secure-store` → Keychain	İşletim sisteminin şifreli kasası, diğer uygulamalar erişemez.
Android	`expo-secure-store` → Keystore	Donanımda tutulan bir anahtarla şifrelenir.
Web	`localStorage`	SecureStore web'de çalışmaz. Sayfadaki her JavaScript okuyabilir (XSS riski). Web asıl hedef olmadığı için bilinçli bir taviz.

Neden `AsyncStorage` değil? Şifrelenmemiş düz bir dosyaya yazar. Token 24 saat boyunca senin yerine giriş yapmayı sağlayan bir anahtar; kasada durmalı. Web için en güvenlisi `httpOnly` cookie, ama o yol backend'de cookie okumayı ve CSRF korumasını geri açmayı gerektiriyor.

`wallet-mobile/src/services/tokenStorage.ts`

```
export const tokenStorage = {
  async get()    { return Platform.OS === 'web' ? localStorage.getItem(KEY) :
SecureStore.getItemAsync(KEY); },
  async set(t)   { Platform.OS === 'web' ? localStorage.setItem(KEY, t) : await
SecureStore.setItemAsync(KEY, t); },
  async remove() { Platform.OS === 'web' ? localStorage.removeItem(KEY) : await
SecureStore.deleteItemAsync(KEY); },
};   // (kısaltılmış; gerçek dosyada localStorage erişim hataları da yakalanıyor)
```

Ekranlar token'ın nerede durduğunu bilmiyor, kararı sadece bu dosya veriyor. Yarın web için cookie'ye geçmek gerekirse sadece bu dosya değişir.

8 Mobil: istekler token ile nasıl gider?

Bütün HTTP istekleri tek bir yerden geçiyor: `apiClient.ts`. Token orada bir değişkende tutuluyor (`setAuthToken`) ve her isteğe ekleniyor.

`wallet-mobile/src/services/api/apiClient.ts`

```
const token = authToken;                                    // istek başında sabitlenir
const headers = { 'Content-Type': 'application/json', Accept: 'application/json' };
if (token) headers.Authorization = `Bearer ${token}`;      // backend'deki filtre bunu okur

const response = await fetch(`${API_BASE_URL}${path}`, { method, headers, body, signal });

// Token gönderildiği halde 401 → oturum geçersiz (süresi dolmuş vb.)
if (response.status === 401 && token) {
  unauthorizedHandler?.();                                  // AuthContext.signOut()
  throw new ApiError('Oturumun sona erdi, lütfen tekrar giriş yap', 401);
}
```

Token'lı 401

Token geçersiz. Oturum kapanır, kullanıcı login ekranına gider.

Token'sız 401

Örneğin login'de yanlış şifre. Oturumla ilgili değil, sadece ekranda hata mesajı gösterilir.

`expenseService` gibi servisler hiçbir değişiklik gerektirmedi: hepsi `apiClient` kullandığı için token'ı otomatik taşıyorlar. Harcama isteğinden `userId` çıkarıldı (`ExpenseRequest` tipi artık izin vermiyor), `CURRENT_USER_ID` sabiti silindi.

9 Mobil: oturum, yönlendirme, ekranlar

9.1 AuthContext: oturumun tek kaynağı

React Context, uygulamanın tepesine konan ve her bileşenin `useAuth()` ile okuyabildiği ortak bir kutu. Token üç yerde tutuluyor ve bu üçü her zaman birlikte güncelleniyor:

Nerede	Ne için
React state (`token`)	Ekranlar ve yönlendirme ("giriş yapılmış mı?")
`apiClient` (`setAuthToken`)	İsteklere başlık eklemek
`tokenStorage`	Uygulama kapanıp açılınca hatırlamak

Fonksiyon	Ne yapıyor
`signIn(req)`	`authService.login` → token'ı kaydet → oturumu aç
`signUp(req)`	`authService.register` → token'ı kaydet → oturumu aç
`signOut()`	Üç yerden de token'ı sil
açılışta	`tokenStorage.get()`; bu sırada `isLoading = true` ve splash ekranı açık kalır

9.2 Yönlendirme: Stack.Protected

`wallet-mobile/src/app/_layout.tsx`

```
<Stack.Screen name="index" />                    {/* token'a göre /home veya /login */}
<Stack.Protected guard={!signedIn}>              {/* sadece oturum YOKKEN */}
  <Stack.Screen name="login" />
  <Stack.Screen name="register" />
</Stack.Protected>
<Stack.Protected guard={signedIn}>               {/* sadece oturum VARKEN */}
  <Stack.Screen name="(drawer)" />
  <Stack.Screen name="expenses/new" />
</Stack.Protected>
```

Ekranlar kendileri yönlendirme yapmıyor. Token dolunca ya da boşalınca kilidi kapanan ekranlar geçmişten silinir ve kullanıcı `index` üzerinden doğru yere gider. Çıkıştan sonra geri tuşu uygulamanın içine dönemez, açık modallar da kapanır.

9.3 Ekranlar

Dosya	Görevi
`LoginPage` + `useLoginForm`	Boş alan kontrolü, 401'de "Kullanıcı adı veya şifre hatalı", yükleniyor durumu
`RegisterPage` + `useRegisterForm`	Backend ile aynı kurallar (Türkçe mesajlar), şifre tekrarı, "kullanıcı adı alınmış" hatası ilgili alanın altında
`AuthScreen`	İki sayfanın ortak iskeleti (logo, başlık, süs daireleri, klavye davranışı)
`PasswordToggle`	Şifreyi göster/gizle ikonu
`LogoutButton`	Sadece `signOut()` çağırıyor, yönlendirmeyi `Stack.Protected` yapıyor

10 Uçtan uca akışlar

Giriş

1	**LoginPage**	Kullanıcı "Giriş yap"a basar → `useLoginForm.submit()` → `signIn()`
2	**apiClient**	`POST /api/auth/login` (token yok)
3	**Filtre**	Başlık yok, hiçbir şey yapmadan geçirir. `/api/auth/**` herkese açık.
4	**AuthService**	`findByUserName` → `matches` → `generateToken` → `{ token }`
5	**AuthContext**	Token'ı SecureStore'a, apiClient'a ve state'e yazar.
6	**Stack.Protected**	Login'in kilidi kapanır → index → `/home`

Harcama ekleme

1	**Mobil**	`POST /api/expenses` + `Authorization: Bearer eyJ…` · gövdede `userId` yok
2	**Filtre**	Token geçerli → `sub = murat` → SecurityContext'e kimlik konur
3	**SecurityConfig**	`authenticated()` kuralı: kutu dolu, geçebilir
4	**ExpenseService**	`currentUserService.getCurrentUser()` → `expense.setUser(murat)` → kaydet

Listeleme

1	**Mobil**	`GET /api/expenses` + token
2	**QueryService**	Filtreler + `user_id = murat'ın id'si` → sadece kendi harcamaları döner

Token'ın süresi dolarsa

1	**Backend**	`parseSignedClaims` süreyi geçmiş görür → kutu boş kalır → 401
2	**apiClient**	Token'lı 401 → `unauthorizedHandler` → `signOut()`
3	**Stack.Protected**	Korumalı ekranlar kapanır → `/login`

Uygulamayı kapatıp açınca

Splash açık kalır → `tokenStorage.get()` → token varsa doğrudan `/home`, yoksa `/login`. Kullanıcı yeniden giriş yapmaz.

11 Yolda karşılaşılan sorunlar

Belirti	Sebep	Çözüm / ders
Uygulama açılırken `ExpenseMapperImpl` ... *Unresolved compilation problems* hatası	Proje hem IntelliJ'de hem VS Code'da açık. VS Code'daki Red Hat Java eklentisi aynı `target/classes` klasörüne Eclipse derleyicisiyle yazıyor ve MapStruct sınıfını bozuyordu.	VS Code'da eklentiyi **Disable (Workspace)** ile kapat, IntelliJ'de **Rebuild Project**. Java için tek IDE.
Test çıktısı görünmüyor; `pom.xml`'e JUnit 4 eklenmiş	Test dosyası `src/main` altındaydı. IntelliJ'in "Add JUnit4" önerisi kabul edilmişti.	Testler `src/test` altına konur, JUnit 5 import edilir (`org.junit.jupiter.api.Test`). "Bağımlılık ekle" önerilerine dikkat.
`loadUserByUsername ... does not override`	`implements UserDetailsService` unutulmuştu.	`@Override` bu hatayı derleme sırasında yakaladı; o olmasa Spring sınıfı sessizce yok sayardı.
Gizli anahtar	Önce yer tutucu, sonra örnekteki değer kopyalanmıştı.	Anahtar sadece senin bildiğin rastgele bir değer olmalı. Komutla kendi değerini üret.
`SecurityConfig`'te çift import	Kopyala-yapıştır	IntelliJ: **Ctrl+Alt+O** (Optimize Imports)

12 Bilinen sınırlar ve sonraki adımlar

Konu	Durum / öneri
Eski harcamalar	Login'den önce eklenen her şey 1 numaralı kullanıcıya ait. Taşımak için: `UPDATE expenses SET user_id = <id> WHERE user_id = 1;`
Token yenileme yok	24 saat sonra kullanıcı tekrar giriş yapar. İleride kısa ömürlü access token + refresh token düşünülebilir.
Süresi dolmuş token ile açılış	Ana sayfa bir an görünüp login'e düşebilir. Çözüm: açılışta `exp` alanını okuyup kontrol etmek.
Web'de localStorage	XSS riski. Web ciddi bir ürüne dönüşürse httpOnly cookie + CSRF koruması.
"Zaten alınmış" mesaj eşlemesi	Mobil, backend'in İngilizce mesajını metne göre Türkçe alana eşliyor. Backend mesajı değişirse `useRegisterForm`'daki `DUPLICATE_ERRORS` sözlüğü de güncellenmeli. Daha sağlam çözüm: backend alan bazlı hata dönsün.
Giriş denemesi sınırı yok	Kaba kuvvet denemelerine karşı login'e hız sınırı (rate limit) eklenebilir.
Gizli anahtar git'te	Geliştirme için kabul edilebilir. Canlıda `JWT_SECRET` ortam değişkeni kullanılmalı.
Roller	`authorities(List.of())` boş. Admin gibi roller eklenirse 403 devreye girer.

Tek cümleyle

Kullanıcı bir kez şifreyle kimliğini kanıtlıyor. Sunucu ona imzalı ve süreli bir token veriyor. Mobil uygulama bu token'ı kasada saklayıp her isteğe ekliyor. Sunucu da "kim?" sorusunun cevabını istemciden değil, sadece bu imzalı token'dan alıyor. Böylece herkes yalnızca kendi verisini görebiliyor.